

PENETRATION TEST WALKTHROUGH

JWT Forgery

백업 파일을 통한 JWT 시크릿 노출과 권한 상승 (Backup File Disclosure & JWT Privilege Escalation)

TARGET problem03
PREPARED FOR 8low8lowmi Team

SEVERITY: HIGH

CWE-347 · CWE-798

STATUS: DRAFT

1. 개요 (Executive Summary)

problem03(관리자 로그인 문제)을 살펴본 결과, 백업 파일을 통해 JWT 서명에 사용되는 시크릿 키가 그대로 노출되는 것을 발견했다.

robots.txt가 /backup/ 경로의 존재를 암시하고 있었고, 그 안의 backup.zip을 열어보니 개발 환경에서 쓰던 config.env 파일이 그대로 들어있었다. 이 파일 안에 JWT 서명 키(JWT_SECRET)가 평문으로 적혀 있었다. 일반 사용자로 로그인해 정상 발급된 JWT를 디코딩해보니 payload 안에 role 값이 그대로 노출되어 있었고, 이 값을 admin으로 바꾼 뒤 유출된 시크릿 키로 재서명하니 서버가 이를 정상적인 관리자 토큰으로 인식했다. 결과적으로 실제 관리자 계정 정보를 전혀 몰라도 로그인 한 번과 파일 두 개(robots.txt, backup.zip)만으로 관리자 권한을 획득할 수 있었다.

2. 공격 시나리오 (Attack Narrative)

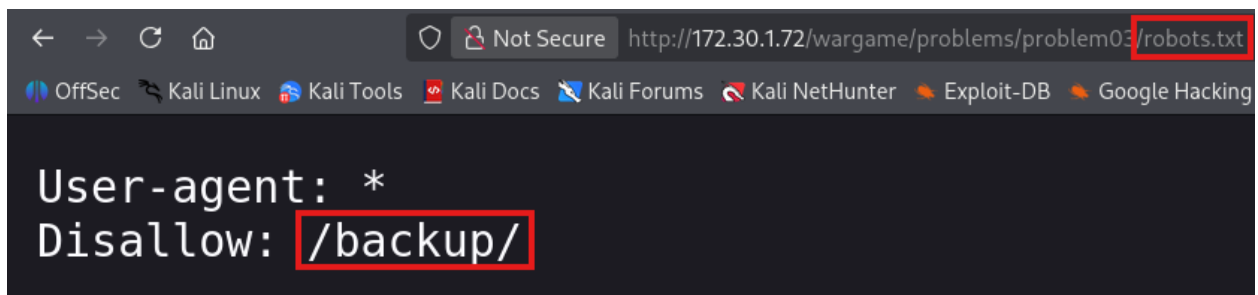
단계	기법	확보한 정보 / 결과
1	robots.txt 확인	/backup/ 경로 노출 확인 (Disallow 항목으로 역으로 위치 암시)
2	backup.zip 다운로드 및 압축 해제	config.env 확보 — JWT_SECRET, ADMIN_PANEL, DEBUG_PAGE 경로 노출 확인
3	회원가입 후 일반 사용자로 로그인	정상 JWT 발급, payload 디코딩 시 role: user 확인
4	JWT payload role 값 변조 (user → admin)	위조된 payload 준비

5	유출된 JWT_SECRET으로 위조 payload 재서명	서명이 유효한 admin 권한 JWT 생성
6	위조 토큰으로 /admin, /api/admin 접근	Admin Panel 접근 성공, 인증 우회 완료
7	Backup 메뉴에서 backup_3.zip 다운로드	압축 해제 후 flag.txt에서 FLAG 확보

3. 상세 재현 절차 (Proof of Concept)

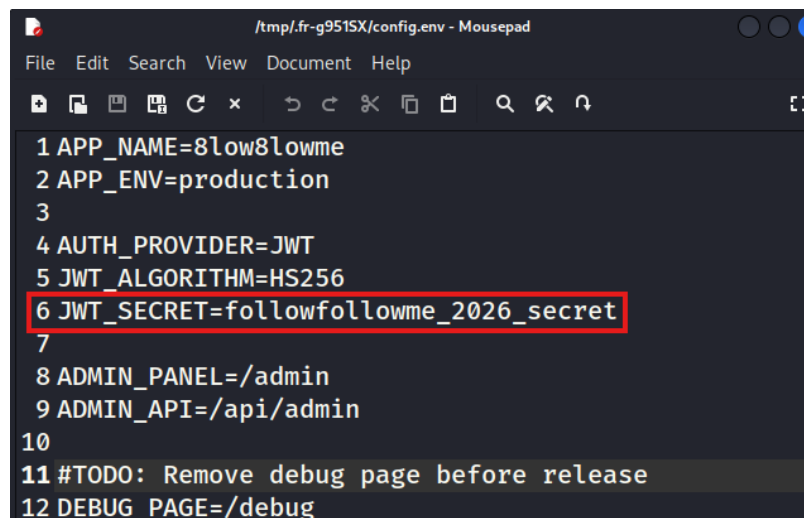
3.1 정찰 — robots.txt 및 백업 경로 확인

problem03 루트의 robots.txt를 확인하면 /backup/ 경로가 Disallow로 지정되어 있다. 크롤러 차단을 위한 설정이지만, 참가자 입장에서 오�히려 "이 경로에 뭔가 있다"는 힌트로 작동한다.



3.2 backup.zip 확보 및 config.env 분석

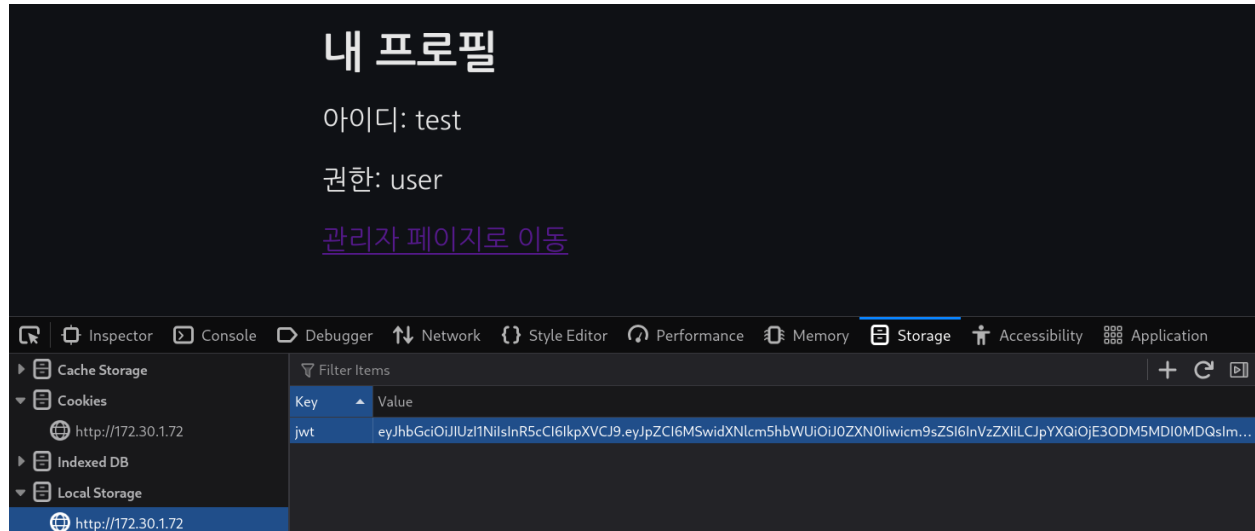
/backup/backup.zip을 다운로드해 압축을 풀면 config.env와 README.txt가 나온다. config.env 안에 JWT 서명에 쓰이는 시크릿 키가 평문으로 남아있다.



3.3 일반 사용자 JWT 구조 확인

회원가입 후 로그인하면 정상적으로 JWT가 발급된다. 이 토큰을 jwt.io 또는 직접 base64url 디코딩해보면 payload 안에 role: user가 그대로 들어있는 것을 확인할 수 있다.

이 상태에서 관리자 페이지로 이동시 권한이 없다고 뜬다.



3.4 payload 변조 및 재서명

JWT 안에 있는 등급 정보(role)를 일반회원(user)에서 관리자(admin)로 고치고, 아까 백업파일에서 찾은 비밀 키로 그 위조된 내용에 다시 도장을 찍는다. PyJWT라는 파이썬 도구를 쓰면 이 과정을 코드 몇 줄로 간단히 할 수 있다.

```
import jwt
import time

secret = 'followfollowme_2026_secret'

now = int(time.time()) # 지금 이 순간의 시간

payload = {
    'id': 1,
    'username': 'test',
    'role': 'admin',
    'iat': now,
    'exp': now + 3600, # 지금부터 1시간 뒤 만료
}

forged_token = jwt.encode(payload, secret, algorithm='HS256')
print(forged_token)
```

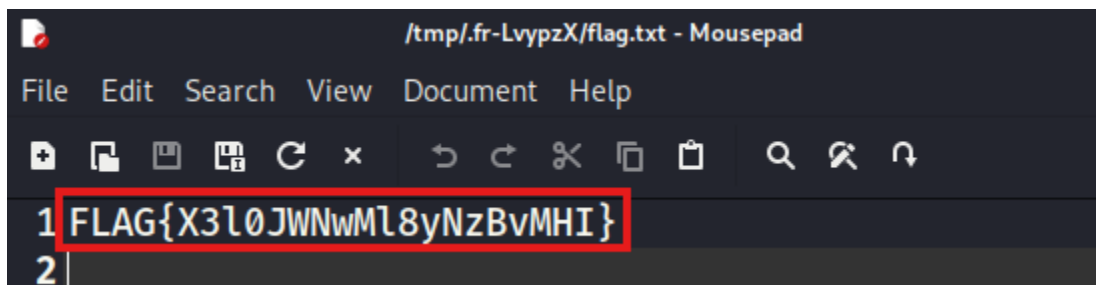
■ 3.5 위조 토큰으로 Admin Panel 접근

브라우저 devtools 콘솔에서 localStorage의 jwt 값을 위조한 토큰으로 교체한 뒤 admin.php를 새로고침 하면, 서버가 서명 검증만으로 role을 신뢰하기 때문에 실제 관리자 계정 정보 없이도 Admin Panel에 접근할 수 있다.



■ 3.6 FLAG 획득

Admin Panel의 Backup 메뉴에서 backup_3.zip을 내려받아 압축을 풀면 flag.txt 안에 최종 FLAG가 들어 있다.



4. 근본 원인 분석 (Root Cause)

- JWT 서명 시크릿(JWT_SECRET)이 개발용 백업 파일(config.env)에 평문으로 저장된 채 운영 서버에 그대로 배포됨
- robots.txt 의 Disallow 설정이 오히려 민감한 경로(/backup/)의 존재를 외부에 암시함
- 서버가 JWT 의 서명 유효성만 검증할 뿐, payload 의 role 값을 DB 의 실제 사용자 상태와 재대조하지 않아 서명만 통과하면 payload 내용을 그대로 신뢰함
- JWT_SECRET 에 대한 로테이션·만료 정책이 없어, 한 번 유출되면 무기한으로 위조 토큰 발급이 가능함